NULL IN THE ORACLE DATABASE

SQL's NULL frequently causes confusion. Although the basic idea of NULL—to represent missing data—is rather simple, there are some peculiarities. You have to use IS NULL instead of = NULL, for example. Moreover the Oracle database has additional NULL oddities, on the one hand because it does not always handle NULL as required by the standard and on the other hand because it has a very "special" handling of NULL in indexes.

The SQL standard does not define NULL as a value but rather as a placeholder for a missing or unknown value. Consequently, no value can be NULL. Instead the Oracle database treats an empty string as NULL:

```
   SELECT     '0 IS NULL???' AS "what is NULL?" FROM dual
    WHERE      0 IS NULL
UNION ALL
   SELECT    '0 is not null' FROM dual
    WHERE     0 IS NOT NULL
UNION ALL
   SELECT '''''  IS NULL???'  FROM dual
    WHERE    '' IS NULL
UNION ALL
   SELECT ''''' is not null' FROM dual
    WHERE    '' IS NOT NULL;

what is NULL?
--------------
0 is not null
'' IS NULL???
```

To add to the confusion, there is even a case when the Oracle database treats NULL as empty string:

```
SELECT dummy
     , dummy || ''
     , dummy || NULL
  FROM dual;

D D D
- - -
X X X
```

Concatenating the DUMMY column (always containing 'X') with NULL should return NULL.

The concept of NULL is used in many programming languages. No matter where you look, an empty string is never NULL…except in the Oracle database. It is, in fact, impossible to store an empty string in a VARCHAR2 field. If you try, the Oracle database just stores NULL.

This peculiarity is not only strange; it is also dangerous. Additionally the Oracle database's NULL oddity does not stop here—it continues with indexing.

INDEXING NULL

The Oracle database does not include rows in an index if all indexed columns are NULL. That means that every index is a partial index—like having a **where** clause:

```
CREATE INDEX idx
          ON tbl (A, B, C, ...)
       WHERE A IS NOT NULL
          OR B IS NOT NULL
          OR C IS NOT NULL
             ...;
```

Consider the EMP_DOB index. It has only one column: the DATE_OF_BIRTH. A row that does not have a DATE_OF_BIRTH value is not added to this index.

```
INSERT INTO employees ( subsidiary_id, employee_id
                      , first_name   , last_name
                      , phone_number)
               VALUES ( ?, ?, ?, ?, ? );
```

The **insert** statement does not set the DATE_OF_BIRTH so it defaults to NULL—hence, the record is not added to the EMP_DOB index. As a consequence, the index cannot support a query for records where DATE_OF_BIRTH IS NULL:

```
SELECT first_name, last_name
  FROM employees
 WHERE date_of_birth IS NULL;
```


EX LIBRIS GHEORGHE GABRIEL SICHIM <GABRIEL.SICHIM@GMAIL.COM>

```
-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("DATE_OF_BIRTH" IS NULL)
```

Nevertheless, the record is inserted into a concatenated index if at least one index column is not NULL:

```
CREATE INDEX demo_null
          ON employees (subsidiary_id, date_of_birth);
```

The above created row is added to the index because the SUBSIDIARY_ID is not NULL. This index can thus support a query for all employees of a specific subsidiary that have no DATE_OF_BIRTH value:

```
SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = ?
   AND date_of_birth IS NULL;

-------------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-------------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    2 |
|  1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    2 |
|* 2 |   INDEX RANGE SCAN           | DEMO_NULL |    1 |    1 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(?)
          AND "DATE_OF_BIRTH" IS NULL)
```

Please note that the index covers the entire **where** clause; all filters are used as access predicates during the INDEX RANGE SCAN.

We can extend this concept for the original query to find all records where DATE_OF_BIRTH IS NULL. For that, the DATE_OF_BIRTH column has to be the leftmost column in the index so that it can be used as access predicate. Although we do not need a second index column for the query itself, we add another column that can never be NULL to make sure the index has all rows.

We can use any column that has a NOT NULL constraint, like SUBSIDIARY_ID, for that purpose.

Alternatively, we can use a constant expression that can never be NULL. That makes sure the index has all rows—even if DATE_OF_BIRTH is NULL.

```
DROP   INDEX emp_dob;
CREATE INDEX emp_dob ON employees (date_of_birth, '1');
```

Technically, this index is a function-based index. This example also disproves the myth that the Oracle database cannot index NULL.



Tip

Add a column that cannot be NULL to index NULL like any value.

NOT NULL Constraints

To index an IS NULL condition in the Oracle database, the index must have a column that can never be NULL.

That said, it is not enough that there are no NULL entries. The database has to be sure there can never be a NULL entry, otherwise the database must assume that the table has rows that are not in the index.

The following index supports the query only if the column LAST_NAME has a NOT NULL constraint:

```
DROP INDEX emp_dob;
CREATE INDEX emp_dob_name
          ON employees (date_of_birth, last_name);

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

```
-------------------------------------------------------------------
|Id |Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 |SELECT STATEMENT             |              |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_NAME |    1 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("DATE_OF_BIRTH" IS NULL)
```

Removing the NOT NULL constraint renders the index unusable for this query:

```
ALTER TABLE employees MODIFY last_name NULL;

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```



TIP

A missing NOT NULL constraint can prevent index usage in an Oracle database—especially for `count(*)` queries.

Besides NOT NULL constraints, the database also knows that constant expressions like in the previous section cannot become NULL.

An index on a user-defined function, however, does not impose a NOT NULL constraint on the index expression:

```
CREATE OR REPLACE FUNCTION blackbox(id IN NUMBER) RETURN NUMBER
DETERMINISTIC
IS BEGIN
   RETURN id;
END;

DROP INDEX emp_dob_name;
CREATE INDEX emp_dob_bb
    ON employees (date_of_birth, blackbox(employee_id));
```

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

The function name BLACKBOX emphasizes the fact that the optimizer has no idea what the function does. We can see that the function passes the input value straight through, but for the database it is just a function that returns a number. The NOT NULL property of the parameter is lost. Although the index must have all rows, the database does not know that so it cannot use the index for the query.

If *you know* that the function never returns NULL, as in this example, you can change the query to reflect that:

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL
   AND blackbox(employee_id) IS NOT NULL;

-------------------------------------------------------------
|Id |Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |            |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_BB |    1 |    2 |
-------------------------------------------------------------
```

The extra condition in the **where** clause is always true and therefore does not change the result. Nevertheless the Oracle database recognizes that you only query rows that must be in the index per definition.

There is, unfortunately, no way to tag a function that never returns NULL but you can move the function call to a virtual column (since 11*g*) and put a NOT NULL constraint on this column.

```
ALTER TABLE employees ADD bb_expression
      GENERATED ALWAYS AS (blackbox(employee_id)) NOT NULL;

DROP   INDEX emp_dob_bb;
CREATE INDEX emp_dob_bb
    ON employees (date_of_birth, bb_expression);
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-------------------------------------------------------------
|Id |Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------
| 0 |SELECT STATEMENT             |            |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_BB |    1 |    2 |
-------------------------------------------------------------
```

The Oracle database knows that some internal functions only return NULL if NULL is provided as input.

```
DROP INDEX emp_dob_bb;

CREATE INDEX emp_dob_upname
    ON employees (date_of_birth, upper(last_name));

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------------
|Id |Operation                    | Name           | Cost |
-----------------------------------------------------------
| 0 |SELECT STATEMENT             |                |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES      |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_UPNAME |    2 |
-----------------------------------------------------------
```

The UPPER function preserves the NOT NULL property of the LAST_NAME column. Removing the constraint, however, renders the index unusable:

```
ALTER TABLE employees MODIFY last_name NULL;

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

Emulating Partial Indexes

The strange way the Oracle database handles NULL in indexes can be used to emulate partial indexes. For that, we just have to use NULL for rows that should not be indexed.

To demonstrate, we emulate the following partial index:

```
CREATE INDEX messages_todo
          ON messages (receiver)
       WHERE processed = 'N'
```

First, we need a function that returns the RECEIVER value only if the PROCESSED value is 'N'.

```
CREATE OR REPLACE
FUNCTION pi_processed(processed CHAR, receiver NUMBER)
RETURN NUMBER
DETERMINISTIC
AS BEGIN
   IF processed IN ('N') THEN
      RETURN receiver;
   ELSE
      RETURN NULL;
   END IF;
END;
/
```

The function must be deterministic so it can be used in an index definition.

Now we can create an index that contains only the rows having PROCESSED='N'.

```
CREATE INDEX messages_todo
          ON messages (pi_processed(processed, receiver));
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

To use the index, you must use the indexed expression in the query:

```
SELECT message
  FROM messages
 WHERE pi_processed(processed, receiver) = ?

-------------------------------------------------------------
|Id | Operation                    | Name          | Cost |
-------------------------------------------------------------
| 0 | SELECT STATEMENT             |               | 5330 |
| 1 |  TABLE ACCESS BY INDEX ROWID| MESSAGES      | 5330 |
|*2 |   INDEX RANGE SCAN           | MESSAGES_TODO | 5303 |
-------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("PI_PROCESSED"("PROCESSED","RECEIVER")=:X)
```

Partial Indexes, Part II

As of release 11*g*, there is a second—equally scary—approach to emulating partial indexes in the Oracle database by using an intentionally broken index partition and the `SKIP_UNUSABLE_INDEX` parameter.

Obfuscated Conditions

The following sections demonstrate some popular methods for obfuscating conditions. Obfuscated conditions are **where** clauses that are phrased in a way that prevents proper index usage. This section is a collection of anti-patterns every developer should know about and avoid.

Date Types

Most obfuscations involve DATE types. The Oracle database is particularly vulnerable in this respect because it has only one DATE type that always includes a time component as well.

It has become common practice to use the TRUNC function to remove the time component. In truth, it does not remove the time but instead sets it to midnight because the Oracle database has no pure DATE type. To disregard the time component for a search you can use the TRUNC function on both sides of the comparison—e.g., to search for yesterday's sales:

```
SELECT ...
  FROM sales
 WHERE TRUNC(sale_date) = TRUNC(sysdate - INTERVAL '1' DAY)
```

It is a perfectly valid and correct statement but it cannot properly make use of an index on SALE_DATE. It is as explained in "Case-Insensitive Search Using UPPER or LOWER" on page 24; TRUNC(sale_date) is something entirely different from SALE_DATE—functions are black boxes to the database.

There is a rather simple solution for this problem: a function-based index.

```
CREATE INDEX index_name
          ON table_name (TRUNC(sale_date))
```

But then you must always use TRUNC(date_column) in the **where** clause. If you use it inconsistently—sometimes with, sometimes without TRUNC—then you need two indexes!

The problem also occurs with databases that have a pure date type if you search for a longer period as shown in the following MySQL query:

```
SELECT ...
  FROM sales
 WHERE DATE_FORMAT(sale_date, "%Y-%M")
     = DATE_FORMAT(now()    , "%Y-%M')
```

The query uses a date format that only contains year and month: again, this is an absolutely correct query that has the same problem as before. However the solution from above does not apply here because MySQL has no function-based indexes.

The alternative is to use an explicit range condition. This is a generic solution that works for all databases:

```
SELECT ...
  FROM sales
 WHERE sale_date BETWEEN quarter_begin(?)
                     AND quarter_end(?)
```

If you have done your homework, you probably recognize the pattern from the exercise about all employees who are 42 years old.

A straight index on SALE_DATE is enough to optimize this query. The functions QUARTER_BEGIN and QUARTER_END compute the boundary dates. The calculation can become a little complex because the **between** operator always includes the boundary values. The QUARTER_END function must therefore return a time stamp just before the first day of the next quarter if the SALE_DATE has a time component. This logic can be hidden in the function.

The examples on the following pages show implementations of the functions QUARTER_BEGIN and QUARTER_END for various databases.

MySQL

```
CREATE FUNCTION quarter_begin(dt DATETIME)
RETURNS DATETIME DETERMINISTIC
RETURN CONVERT
       (
         CONCAT
         ( CONVERT(YEAR(dt),CHAR(4))
         , '-'
         , CONVERT(QUARTER(dt)*3-2,CHAR(2))
         , '-01'
         )
       , datetime
       );

CREATE FUNCTION quarter_end(dt DATETIME)
RETURNS DATETIME DETERMINISTIC
RETURN DATE_ADD
       ( DATE_ADD ( quarter_begin(dt), INTERVAL 3 MONTH )
       , INTERVAL -1 MICROSECOND);
```

Oracle Database

```
CREATE FUNCTION quarter_begin(dt IN DATE)
RETURN DATE
AS
BEGIN
   RETURN TRUNC(dt, 'Q');
END;
/

CREATE FUNCTION quarter_end(dt IN DATE)
RETURN DATE
AS
BEGIN
   -- the Oracle DATE type has seconds resolution
   -- subtract one second from the first
   -- day of the following quarter
   RETURN TRUNC(ADD_MONTHS(dt, +3), 'Q')
        - (1/(24*60*60));
END;
/
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

PostgreSQL

```
CREATE FUNCTION quarter_begin(dt timestamp with time zone)
RETURNS timestamp with time zone AS $$
BEGIN
    RETURN date_trunc('quarter', dt);
END;
$$ LANGUAGE plpgsql;

CREATE FUNCTION quarter_end(dt timestamp with time zone)
RETURNS timestamp with time zone AS $$
BEGIN
   RETURN   date_trunc('quarter', dt)
          + interval '3 month'
          - interval '1 microsecond';
END;
$$ LANGUAGE plpgsql;
```

SQL Server

```
CREATE FUNCTION quarter_begin (@dt DATETIME )
RETURNS DATETIME
BEGIN
  RETURN DATEADD (qq, DATEDIFF (qq, 0, @dt), 0)
END
GO

CREATE FUNCTION quarter_end (@dt DATETIME )
RETURNS DATETIME
BEGIN
  RETURN DATEADD
         ( ms
         , -3
         , DATEADD(mm, 3, dbo.quarter_begin(@dt))
         );
END
GO
```

You can use similar auxiliary functions for other periods—most of them will be less complex than the examples above, especially when using than or greater equal to (>=) and less than (<) conditions instead of the **between** operator. Of course you could calculate the boundary dates in your application if you wish.



Tip

Write queries for continuous periods as explicit range condition. Do this even for a single day—e.g., for the Oracle database:

```
    sale_date >= TRUNC(sysdate)
AND sale_date <  TRUNC(sysdate + INTERVAL '1' DAY)
```

Another common obfuscation is to compare dates as strings as shown in the following PostgreSQL example:

```
SELECT ...
  FROM sales
 WHERE TO_CHAR(sale_Date, 'YYYY-MM-DD') = '1970-01-01'
```

The problem is, again, converting DATE_COLUMN. Such conditions are often created in the belief that you cannot pass different types than numbers and strings to the database. Bind parameters, however, support all data types. That means you can for example use a `java.util.Date` object as bind parameter. This is yet another benefit of bind parameters.

If you cannot do that, you just have to convert the search term instead of the table column:

```
SELECT ...
  FROM sales
 WHERE sale_date = TO_DATE('1970-01-01', 'YYYY-MM-DD')
```

This query can use a straight index on SALE_DATE. Moreover it converts the input string only once. The previous statement must convert all dates stored in the table before it can compare them against the search term.

Whatever change you make—using a bind parameter or converting the other side of the comparison—you can easily introduce a bug if SALE_DATE has a time component. You must use an explicit range condition in that case:

```
SELECT ...
  FROM sales
 WHERE sale_date >= TO_DATE('1970-01-01', 'YYYY-MM-DD')
   AND sale_date <  TO_DATE('1970-01-01', 'YYYY-MM-DD')
                  + INTERVAL '1' DAY
```

Always consider using an explicit range condition when comparing dates.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

LIKE on Date Types

The following obfuscation is particularly tricky:

```
sale_date LIKE SYSDATE
```

It does not look like an obfuscation at first glance because it does not use any functions.

The `LIKE` operator, however, enforces a string comparison. Depending on the database, that might yield an error or cause an implicit type conversion on both sides. The "Predicate Information" section of the execution plan shows what the Oracle database does:

```
filter( INTERNAL_FUNCTION(SALE_DATE)
   LIKE TO_CHAR(SYSDATE@!))
```

The function `INTERNAL_FUNCTION` converts the type of the `SALE_DATE` column. As a side effect it also prevents using a straight index on `DATE_COLUMN` *just as any other function would*.

Numeric Strings

Numeric strings are numbers that are stored in text columns. Although it is a very bad practice, it does not automatically render an index useless if you consistently treat it as string:

```
SELECT ...
  FROM ...
 WHERE numeric_string = '42'
```

Of course this statement can use an index on NUMERIC_STRING. If you compare it using a number, however, the database can no longer use this condition as an access predicate.

```
SELECT ...
  FROM ...
 WHERE numeric_string = 42
```

Note the missing quotes. Although some database yield an error (e.g. PostgreSQL) many databases just add an implicit type conversion.

```
SELECT ...
  FROM ...
 WHERE TO_NUMBER(numeric_string) = 42
```

It is the same problem as before. An index on NUMERIC_STRING cannot be used due to the function call. The solution is also the same as before: do not convert the table column, instead convert the search term.

```
SELECT ...
  FROM ...
 WHERE numeric_string = TO_CHAR(42)
```

You might wonder why the database does not do it this way automatically? It is because converting a string to a number always gives an unambiguous result. This is not true the other way around. A number, formatted as text, can contain spaces, punctation, and leading zeros. A single value can be written in many ways:

```
42
042
0042
00042
...
```

The database cannot know the number format used in the NUMERIC_STRING column so it does it the other way around: the database converts the strings to numbers—this is an unambiguous transformation.

The TO_CHAR function returns only one string representation of the number. It will therefore only match the first of above listed strings. If we use TO_NUMBER, it matches all of them. That means there is not only a performance difference between the two variants but also a semantic difference!

Using numeric strings is generally troublesome: most importantly it causes performance problems due to the implicit conversion and also introduces a risk of running into conversion errors due to invalid numbers. Even the most trivial query that does not use any functions in the **where** clause can cause an abort with a conversion error if there is just one invalid number stored in the table.



Tip

Use numeric types to store numbers.

Note that the problem does not exist the other way around:

```
SELECT ...
  FROM ...
 WHERE numeric_number = '42'
```

The database will consistently transform the string into a number. It does not apply a function on the potentially indexed column: a regular index will therefore work. Nevertheless it is possible to do a manual conversion the wrong way:

```
SELECT ...
  FROM ...
 WHERE TO_CHAR(numeric_number) = '42'
```

Combining Columns

This section is about a popular obfuscation that affects concatenated indexes.

The first example is again about date and time types but the other way around. The following MySQL query combines a data and a time column to apply a range filter on both of them.

```
SELECT ...
  FROM ...
 WHERE ADDTIME(date_column, time_column)
     > DATE_ADD(now(), INTERVAL -1 DAY)
```

It selects all records from the last 24 hours. The query cannot use a concatenated index on (DATE_COLUMN, TIME_COLUMN) properly because the search is not done on the indexed columns but on derived data.

You can avoid this problem by using a data type that has both a date and time component (e.g., MySQL DATETIME). You can then use this column without a function call:

```
SELECT ...
  FROM ...
 WHERE datetime_column
     > DATE_ADD(now(), INTERVAL -1 DAY)
```

Unfortunately it is often not possible to change the table when facing this problem.

The next option is a function-based index if the database supports it—although this has all the drawbacks discussed before. When using MySQL, function-based indexes are not an option anyway.

It is still possible to write the query so that the database can use a concatenated index on DATE_COLUMN, TIME_COLUMN with an access predicate—at least partially. For that, we add an extra condition on the DATE_COLUMN.

```
 WHERE ADDTIME(date_column, time_column)
     > DATE_ADD(now(), INTERVAL -1 DAY)
   AND date_column
    >= DATE(DATE_ADD(now(), INTERVAL -1 DAY))
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

The new condition is absolutely redundant but it is a straight filter on DATE_COLUMN that can be used as access predicate. Even though this technique is not perfect, it is usually a good enough approximation.



Tip

Use a redundant condition on the most significant column when a range condition combines multiple columns.

For PostgreSQL, it's preferable to use the row values syntax described on page 151.

You can also use this technique when storing date and time in text columns, but you have to use date and time formats that yields a chronological order when sorted lexically—e.g., as suggested by ISO 8601 (YYYY-MM-DD HH:MM:SS). The following example uses the Oracle database's TO_CHAR function for that purpose:

```
SELECT ...
  FROM ...
 WHERE date_string || time_string
     > TO_CHAR(sysdate - 1, 'YYYY-MM-DD HH24:MI:SS')
   AND date_string
    >= TO_CHAR(sysdate - 1, 'YYYY-MM-DD')
```

We will face the problem of applying a range condition over multiple columns again in the section entitled "Paging Through Results". We'll also use the same approximation method to mitigate it.

Sometimes we have the reverse case and might want to obfuscate a condition intentionally so it cannot be used anymore as access predicate. We already looked at that problem when discussing the effects of bind parameters on LIKE conditions. Consider the following example:

```
SELECT last_name, first_name, employee_id
  FROM employees
 WHERE subsidiary_id = ?
   AND last_name LIKE ?
```

Assuming there is an index on SUBSIDIARY_ID and another one on LAST_NAME, which one is better for this query?

Without knowing the wildcard's position in the search term, it is impossible to give a qualified answer. The optimizer has no other choice than to "guess". If *you know* that there is always a leading wildcard, you can obfuscate the LIKE condition intentionally so that the optimizer can no longer consider the index on LAST_NAME.

```
SELECT last_name, first_name, employee_id
  FROM employees
 WHERE subsidiary_id = ?
   AND last_name || '' LIKE ?
```

It is enough to append an empty string to the LAST_NAME column. This is, however, an option of last resort. Only do it when absolutely necessary.

Smart Logic

One of the key features of SQL databases is their support for ad-hoc queries: new queries can be executed at any time. This is only possible because the query optimizer (query planner) works at runtime; it analyzes each statement when received and generates a reasonable execution plan immediately. The overhead introduced by runtime optimization can be minimized with bind parameters.

The gist of that recap is that databases are optimized for dynamic SQL—so use it if you need it.

Nevertheless there is a widely used practice that avoids dynamic SQL in favor of static SQL—often because of the "dynamic SQL is slow" myth. This practice does more harm than good if the database uses a shared execution plan cache like DB2, the Oracle database, or SQL Server.

For the sake of demonstration, imagine an application that queries the EMPLOYEES table. The application allows searching for subsidiary id, employee id and last name (case-insensitive) in any combination. It is still possible to write a single query that covers all cases by using "smart" logic.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>